COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	9/17/2026	**DATE SUBMITTED:**	9/24/2026
NAME:	Al Twain D. Pasion	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. 9 **Trees ADT**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Create C++ code to implement a general tree, binary tree and binary search tree.

Task 1

CODE

header file

```
#ifndef GENERALTREE_H
#define GENERALTREE_H

#include <iostream>

const int MAX_CHILDREN = 10;   // max number of children per node

template <typename T>
class TreeNode {
private:
    T key;
    TreeNode<T>* children[MAX_CHILDREN];   // array of child pointers
    int childCount;                        // how many children are used

public:
    // Constructor
    TreeNode(T value);

    // Destructor
    ~TreeNode();

    // Tree building / checking
    TreeNode<T>* addChild(T value);
    bool isLeaf();

    // Traversing algorithms
    void preOrder();
    void inOrder();
    void postOrder();
};
```

```cpp
template <typename T>
TreeNode<T>::TreeNode(T value) {
    key = value;
    childCount = 0;
    for (int i = 0; i < MAX_CHILDREN; i++) {
        children[i] = nullptr;
    }
}

template <typename T>
TreeNode<T>::~TreeNode() {
    for (int i = 0; i < childCount; i++) {
        delete children[i];
    }
}

// Adds a child (after the existing children) and returns it
template <typename T>
TreeNode<T>* TreeNode<T>::addChild(T value) {
    if (childCount >= MAX_CHILDREN) {
        std::cout << "Cannot add more children to " << key << std::endl;
        return nullptr;
    }
    children[childCount] = new TreeNode<T>(value);
    childCount++;
    return children[childCount - 1];
}
```

```cpp
// Returns true if the node has no children
template <typename T>
bool TreeNode<T>::isLeaf() {
    return childCount == 0;
}

// Pre-order: Root -> Left -> Right
// first child is Left, the remaining children are Right
template <typename T>
void TreeNode<T>::preOrder() {
    std::cout << key << " ";
    for (int i = 0; i < childCount; i++) {
        children[i]->preOrder();
    }
}

// In-order: Left -> Root -> Right
template <typename T>
void TreeNode<T>::inOrder() {
    if (isLeaf()) {
        std::cout << key << " ";
        return;
    }
    children[0]->inOrder();
    std::cout << key << " ";
    for (int i = 1; i < childCount; i++) {
        children[i]->inOrder();
    }
}
```

```cpp
// Post-order: Left -> Right -> Root
template <typename T>
void TreeNode<T>::postOrder() {
    for (int i = 0; i < childCount; i++) {
        children[i]->postOrder();
    }
    std::cout << key << " ";
}
```

main file

```cpp
#include <iostream>
#include "GeneralTree.h"

int main() {
    TreeNode<char> A( value: 'A');

    // Children of A
    A.addChild('B');
    A.addChild('C');
    TreeNode<char>* D = A.addChild('D');
    TreeNode<char>* E = A.addChild('E');
    TreeNode<char>* F = A.addChild('F');
    TreeNode<char>* G = A.addChild('G');

    // D -> H
    D->addChild('H');

    // E -> I, J
    E->addChild('I');
    TreeNode<char>* J = E->addChild('J');

    // J -> P, Q
    J->addChild('P');
    J->addChild('Q');

    // F -> K, L, M
    F->addChild('K');
    F->addChild('L');
    F->addChild('M');

    // G -> N
    G->addChild('N');

    std::cout << "Pre-order:  ";
    A.preOrder();
    std::cout << "\nIn-order:   ";
    A.inOrder();
    std::cout << "\nPost-order: ";
    A.postOrder();
    std::cout << std::endl;

    return 0;
}
```

Task 2

Node	*Height*	*Depth*
A	*3*	*0*
B	*0*	*1*
C	*0*	*1*
D	*1*	*1*
E	*2*	*1*
F	*1*	*1*
G	*1*	*1*
H	*0*	*2*
I	*0*	*2*
J	*1*	*2*
K	*0*	*2*
L	*0*	*2*
M	*0*	*2*
N	*0*	*2*
P	*0*	*3*
Q	*0*	*3*

ILO B: Create C++ code for implementation of tree traversal methods such as pre-order, in-order and post-order traversal.

Task 3.1

Pre-Order	A B C D H E I J P Q F K L M G N
Post-Order	B C H D I P Q J E K L M F N G A
In-Order	B A C H D I E P J Q K F L M N G

Task 3.2

OUTPUT

```
Pre-order:  A B C D H E I J P Q F K L M G N
In-order:   B A C H D I E P J Q K F L M N G
Post-order: B C H D I P Q J E K L M F N G A
```

OBSERVATION:
Comparing the two outputs, the one by hand and the one by code are the same. It was a bit challenging to understand how pre-order, post-order, and in-order tree traversing works in a non-binary tree structure. The ABC Tree diagram given was not binary at all. This was a hard task.

Task 3.3

UNABLE TO ACCOMPLISH

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Solve given problems using the tree data structure's implementation in C++

Step 1: Implement a binary search tree that will take the following values: 2, 3, 9, 18, 0, 1, 4, 5.

CODE

```
int main() {

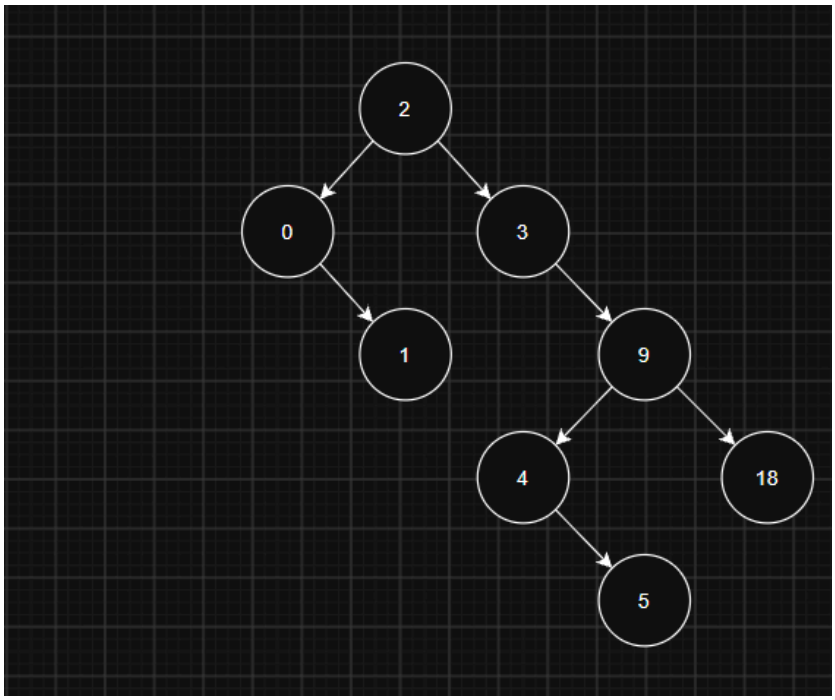
   BinarySearchTree<int> tree( value: 2);

   tree.insert( value: 3);
   tree.insert( value: 9);
   tree.insert( value: 18);
   tree.insert( value: 0);
   tree.insert( value: 1);
   tree.insert( value: 4);
   tree.insert( value: 5);

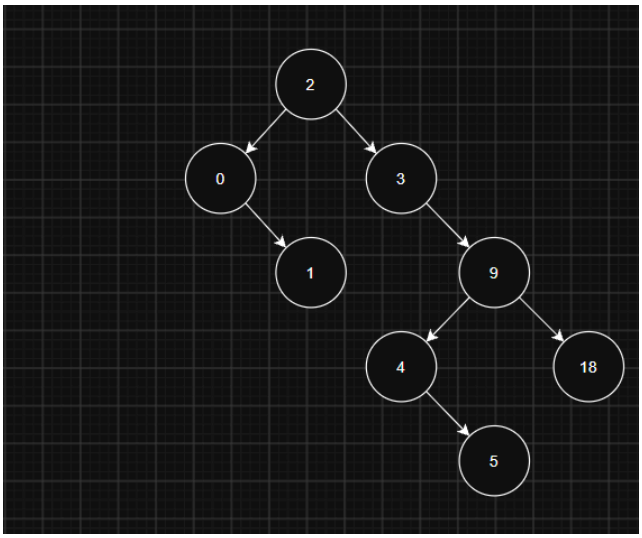
   return 0;
}
```

Step 2: Create a diagram to show the tree after all values have been inserted. Then, with the use of visual aids (like arrows and numbers) indicate the traversal order for in-order, pre-order and post-order traversal on the diagram.

TREE DIAGRAM

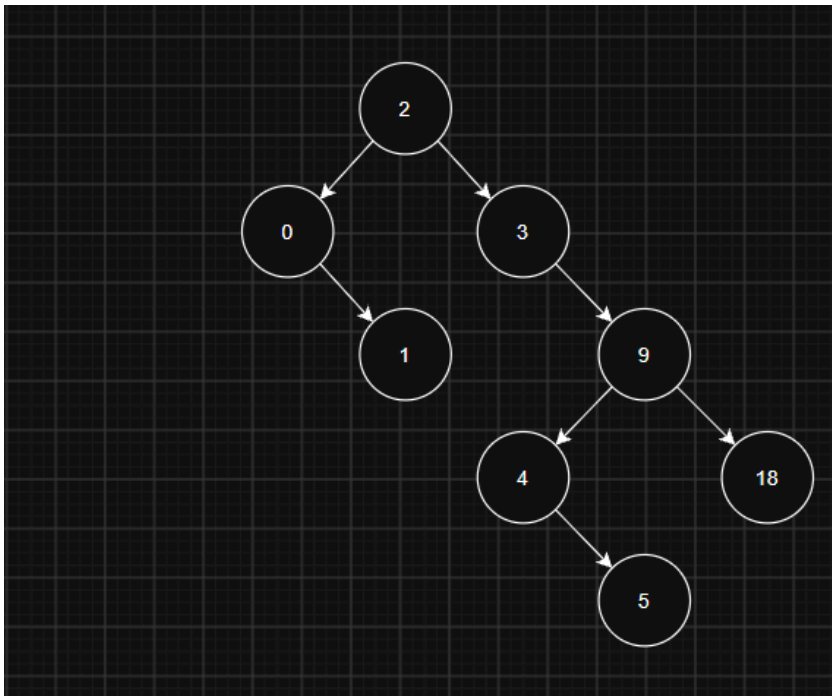


TREE DIAGRAM (IN-ORDER)



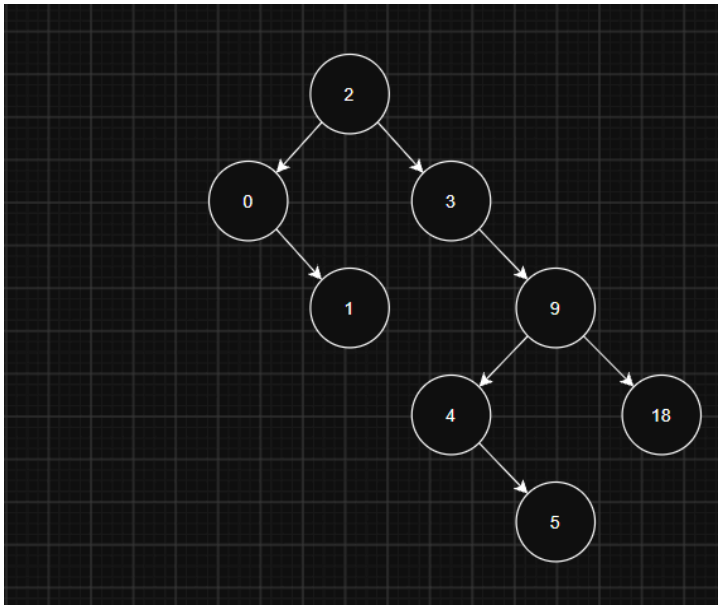
0 -> 1 -> 2 -> 3 -> 4 -> 5 -> 9 -> 18

TREE DIAGRAM (PRE-ORDER)



2 -> 0 -> 1 -> 3 -> 9 -> 4 -> 5 -> 18

TREE DIAGRAM (POST-ORDER)



1 -> 0 -> 5 -> 4 -> 18 -> 9 -> 3 -> 2

Step 3: Compare the different traversal methods. In-order traversal was performed with what function?

CODE

```
template <typename T>
void BinarySearchTree<T>::inOrder() {
    if (left != nullptr) {
        left->inOrder();
    }
    std::cout << key << " ";
    if (right != nullptr) {
        right->inOrder();
    }
}
```

OUTPUT

```
In-order:   0 1 2 3 4 5 9 18
Left -> Root -> Right
```

Given the same input values above, what is the output with different traversal methods? For each output below, indicate your observation: is the output different from the pre-order and post-order traversal that you indicated in the diagrams shown in item #2.

Pre-order Traversal

CODE

```
void BinarySearchTree<T>::preOrder() {
    std::cout << key << " ";
    if (left != nullptr) {
        left->preOrder();
    }
    if (right != nullptr) {
        right->preOrder();
    }
}
```

OUTPUT

```
Pre-order:  2 0 1 3 9 4 5 18
Root -> Left -> Right
```

OBSERVATION: It's exactly the same. Both the manual and the code output.

Post-order Traversal

CODE

```
template <typename T>
void BinarySearchTree<T>::postOrder() {
    if (left != nullptr) {
        left->postOrder();
    }
    if (right != nullptr) {
        right->postOrder();
    }
    std::cout << key << " ";
}
```

OUTPUT

```
Post-order: 1 0 5 4 18 9 3 2
Left -> Right -> Root
```

OBSERVATION: It's exactly the same. Both the manual and the code output.

CONCLUSION
This was a challenging Hands-On-Activity, specifically the procedure. The lessons taught (tree traversal, and etc.) were for a “Binary” Search tree, whereas a node only has one left and one right child. But the Procedure contained a non-binary tree structure. The “A” and “F” nodes have more than two children on the left and/or right. The supplementary was understandable. I applied the tree-traversal methods taught and it was successful. Unfortunately, I did not accomplish Task 3.1 in the **ILO B**. It was hard to understand and apply, especially on a general tree diagram. Overall, this HOA was very difficult. Maybe because I’m unfamiliar with the general tree diagram, and how the tree traversals actually work with such a structure. I’ll make a comeback.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of Concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Explanation on how Tree Traversals (pre,post, and in-order) work on a General Tree Diagram**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.’s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.